

2차 방문 v6.1a 따라하기 (안심구역 · 관악만 분석하는 날)

컬러로 인쇄하세요 (검은 입력 상자·빨간 번호·초록/빨강 칸을 색으로 구분합니다).

어느 안내서를 쓰나: 작업 폴더에 `hbi_code_bundle_v6.1a.txt` 가 있으면 이 안내서. v6.1a 는 자료가 어떻게 생겼든 코드를 고치지 않고 맞춥니다. 자료 맨 위 폴더 하나만 알려 주면 `setup` 이 나머지를 찾습니다.

오늘은 관악만 분석하는 날: 지금 받은 수치지형도·DEM 은 관악구 11도엽뿐입니다 (나머지 4개 구는 메일로 요청 함). `setup` 이 자료가 덮는 대상 구: 관악구. 이 구만 분석할까요? 를 물으면 `Enter`. 그러면 결과는 관악구만 나오고, 선정지 5곳 중 봉천동 1곳만 계산됩니다 (정상). 화면·결과 파일에 대상 5개 구 중 1개 구만 자료 있음 이 남습니다. 관악(11도엽) 밖의 구간·시설·등시선 점은 화면에 → 길 자료 범위 밖으로 보고 건너뛴 이 붙은 줄 이 나오고 결과에 줄이 생기지 않습니다. 관악 밖 선정지는 결과에 건물 0 인 빈 줄로 남습니다 (모두 정상). 나머지 구 자료가 들어오면 `setup` 부터 다시 하면 원래 5개 구로 돌아갑니다.

읽는 법 — 무엇을 입력하나

- "입력" 이 붙은 검은 상자 안의 글자만 명령창에 그대로 칩니다. 위의 "하는 일" 은 설명이라 치지 않습니다.
- 문장이나 표 속의 `python check.py` 처럼 검은 칸도 입력할 명령입니다. 회색 칸은 화면이나 파일에서 찾아볼 글자라 치지 않습니다.
- 안심구역 PC 의 자료·번들 폴더는 매번 `C:\Users\user\Desktop\박수범` 입니다 (안내서의 명령은 모두 이 주소 기준). 그림 속 빨간 번호 ①②③ 은 그 장의 글에서 "그림 ①" 처럼 가리키는 곳입니다.

순서	할 일	입력
단계 0~2	준비 → 명령창 → 번들 풀기	<code>python</code> <code>hbi_code_bundle_v6.1a.txt</code>
단계 3	준비 도우미: 자료를 찾아 설정을 채움 (질문에는 <code>Enter</code> , 관악만 분석할지도 <code>Enter</code>)	<code>python setup.py</code>
단계 4	출발 전 점검 (멈추면 질문에 <code>Enter</code> 하거나 <code>mapping.txt</code> 몇 줄 고치기)	<code>python check.py</code>
단계 5	분석: <code>check</code> 가 고른 방식 + "모든 건물을 집"(all) 두 가지를 한 번에 (멈추면 그 번호부터: 단계 6)	<code>python run_all.py --modes</code> <code>auto,all</code>
단계 7	실측 좌표 넣고 시설 효과·추가 분석	<code>python run_all.py --from 09</code> 등
단계 8	반출: <code>output</code> 폴더 하나 (<code>work</code> 는 반출 안 함)	

꼭 지킬 것: 받은 자료 폴더는 옮기거나 지우지 않습니다 (쓸 폴더는 `setup` 이 골라 적어 둠) · `EDGE_BUFFER` 는 바꾸지 않습니다 · 반출은 `output` 만

단계 0. 준비물과 물어볼 것

이 단계에서 할 일

자료와 반입 파일이 늘 쓰던 폴더에 있는지 담당자에게 확인합니다.

1. 챙길 것: 신분증, 이 안내서와 카드, 현장실측 기록지(토요일에 적은 것), 펜
2. 담당자에게 아래를 묻고 오른쪽 칸에 ○/× 를 적습니다.

물어볼 것	○ / ×
① C:\Users\user\Desktop\박수범 에 자료(수치지형도·DEM·필지·상호제공)가 그대로 있나	
② 같은 폴더에 hbi_code_bundle_v6.1a.txt 가 들어왔나	
③ building_register.csv (건축물대장) 가 같은 폴더에 들어왔나	
④ 종로·중·광진·강서구 수치지형도·DEM 이 새로 들어왔나 (메일로 요청한 것)	

✓ 성공하면

①② 가 ○

✕ 이상하면

자료 폴더가 바뀌었으면 새 주소를 받아 적음 (setup 에서 그 주소를 칩니다). ④ 가 ○ 이면 setup 이 관악만 분석할지 묻지 않고 5개 구로 갑니다 (그대로 Enter)

단계 1. 명령창 열기

이 단계에서 할 일

명령창(글자로 컴퓨터에 지시하는 검은 창)을 엽니다.

1. 화면 왼쪽 아래 **시작 단추** → 키보드로 **OSGeo4W** → 목록의 **OSGeo4W Shell** 클릭(그림 ①).
2. 입력 상자 속 명령은 검은 창에 그대로 타자로 치고 Enter. 맨 아래에 `C:\Users\user\Desktop\박수범` > 처럼 **폴더 주소와 >** 가 다시 나타나야 다음 명령을 칩니다.
3. **폴더 이름은 앞 몇 글자만 치고 Tab** 을 누르면 나머지가 채워집니다 (이번 안내서의 긴 주소는 모두 Tab 으로).



화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

명령을 치기 전에 한/영 키로 영어 입력 상태인지 확인합니다. 한글 상태면 `python` 이 `파이썬` 처럼 입력됩니다. 그렇게 쳤으면 Backspace 로 지우고 한/영 키를 한 번 누른 뒤 다시 칩니다.

실행 중에는 검은 창 안을 클릭하지 않습니다. 클릭하면 "선택" 상태가 되어 멈춥니다. 창 맨 위 제목 앞에 **선택** 이 보이면 Esc. 마우스 휠로 올려 보는 것은 괜찮습니다.

✓ 성공하면

검은 창이 뜨고 제목에 "OSGeo4W Shell"

✗ 이상하면

목록에 없으면 담당자에게 "QGIS 의 OSGeo4W Shell 이 어디 있나요?"

단계 2. v6.1a 번들 풀기

이 단계에서 할 일

자료 폴더로 가서 v6.1a 번들을 풀고 hbi6 폴더로 들어갑니다. 1차 때 쓴 hbi 폴더(v5)는 그대로 둡니다 (v6.1a 는 hbi6 폴더에 풀림).

1. 첫 번째 상자: 주소가 길면 `C:\Users\user\Desktop\` 까지 치고 `박` + `Tab`.
2. 두 번째 상자: 완료: 파일 47개 가 나오면 됩니다(그림 ①).
3. 세 번째 상자: 맨 아래가 `...\hbi6>` 로 바뀝니다(그림 ②).
4. 건축물대장(`building_register.csv`)이 ② 폴더에 있으면 그대로 둡니다. setup·분석이 알아서 찾습니다.

하는 일 자료·번들 폴더로 옮겨 갑니다

입력 `cd /d C:\Users\user\Desktop\박수범`

하는 일 v6.1a 번들을 풀어 hbi6 폴더를 만듭니다

입력 `python hbi_code_bundle_v6.1a.txt`

하는 일 hbi6 폴더 안으로 들어갑니다

입력 `cd hbi6`

OSGeo4W Shell

```
C:\Users\user\Desktop\박수범>python hbi_code_bundle_v6.1a.txt
```

```
완료: 파일 47개 → C:\Users\user\Desktop\박수범 ①
```

```
다음: cd hbi6 → python setup.py (README.md 참고)
```

```
C:\Users\user\Desktop\박수범>cd hbi6
```

```
C:\Users\user\Desktop\박수범\hbi6> ②
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

① 완료: 파일 47개, ② 맨 아래가

```
C:\Users\user\Desktop\박수범\hbi6>
```

✗ 이상하면

'python'은(는) 내부 또는 외부 명령... → 단계 9

단계 3. 준비 도우미 (setup) ① 자료 찾기

이 단계에서 할 일

자료 맨 위 폴더를 알려 주면, 하위 폴더를 모두 훑어 자료를 찾습니다. 질문이 나오면 Enter (추천값으로 진행).

1. 아래 상자를 칩니다. 첫 질문 자료 폴더 = C:\Users\user\Desktop\박수범 에 Enter (기본값). 그 폴더가 없을 때만 주소를 직접 칩니다.
2. zip 파일이 있으면 풀까요? 를 묻습니다 → Enter (받은 폴더는 그대로 두고 hbi6\work\unzipped 에 풀립니다).
3. 그림 ① 에서 [있음] · [없음] 을 봅니다. 뒤에 → 결과 약화 나 → 괜찮음 이 붙은 줄은 그대로 계속합니다 (1:5,000 자료에는 보도중심선·버스정류장이 없음). → 분석 불가 가 보이면 멈추고 담당자에게 그 자료를 묻습니다.
4. 그림 ② DEM 5m·1m 와 필지 개수가 0 이 아닌지 봅니다.

하는 일 자료를 찾아 설정(config.py·mapping.txt)을 채웁니다 (자료는 읽기만 함)

입력 python setup.py

setup 앞부분 (리허설: 가짜 관악 자료라 파일 수가 실제와 다름)

```
OSGeo4W Shell

=== 1. 폴더 훑기 ===
C:\Users\user\Desktop\박수범
.shp 29개, .csv 4개, .img 2개, .txt 1개

=== 3. 자료 판별 ===
[없음] 보도중심선(sidewalk_cl): 파일 0개 → 결과 약화 ①
[있음] 도로중심선(road_cl): 파일 11개 ①
[있음] 계단(stairs): 파일 1개
[있음] 건물(building): 파일 4개

...
DEM: 5m 1개, 1m 1개 ②
필지: shp 1개 중 서울 1개 ②
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

① 도로중심선·건물 [있음], ② 0 이 아님

✗ 이상하면

!! 수치지형도가 dxf 로만 있습니다 → 담당자에게 수치지형도 shp 판 요청. 폴더가 없습니다 → 주소 확인 (Tab 으로 다시)

단계 3-2. 준비 도우미 (setup) ② 폴더·범위·칸

이 단계에서 할 일

setup 이 고른 폴더와 분석 범위를 확인하고, 질문에는 Enter.

- 그림 ① 표: 수치지형도 도엽 폴더마다 **겹치는 구와 쓰기 O/x** 를 보여 줍니다. 오늘은 11개 폴더가 모두 **O** ... → **관악구** 입니다. **Enter**.
- 그림 ② **관악만 분석할지 묻는 줄**: 자료가 덮는 대상 구: **관악구**. 이 구만 분석할까요? → **Enter (= 예)**. 바로 아래 → 대상 5개 구 중 1개 구만 자료 있음 이 나오면 됩니다. 나머지 구 자료가 들어온 날(단계 0 ④ 가 O)에는 이 질문이 나오지 않습니다.
- 그림 ③ **AREA_BB0X** 줄: setup 이 정한 분석 범위와 **이유**입니다. 숫자는 고치지 않습니다.
- 그림 ④ 칸 표: 1:5,000 자료는 칸 이름이 한글입니다 (용도·종류·층수, 도로의 도로구분·자동차전용). ? 는 "못 찾음" 입니다.
- 그림 ⑤ 쓸 수 있는 방식을 보이고 **auto**(check 가 연결률로 고름)를 묻습니다 → **Enter**. 상호제공 CSV 를 찾으면 config 에 넣을지 묻습니다 → **Enter**. 마지막에 다음: **python check.py** 가 나오면 끝.

setup 뒷부분 (가운데 줄임)

```
OSGeo4W Shell
수치지형도가 하위 폴더 11개로 나뉘어 있습니다. 대상 구·엽 구와 겹치는 폴더만 쓰자고 제안합니다:
번호   쓰기   폴더   → 겹치는 구
1      O      37612007_2024_00000357077318   → 관악구 1
2      O      37612008_2024_00000357077318   → 관악구

...

11     O      37612030_2024_00000357077328   → 관악구
Enter = 이대로 / 번호(예: 3 7) = O·x 바꾸기:
→ 쓸 폴더 11개 (map_folders 에 저장, 폴더는 옮기거나 지우지 않음)
대상 구 자료 범위: 종로구 0%, 중구 0%, 관악구 100%, 광진구 0%, 강서구 0%
자료가 덮는 대상 구: 관악구. 이 구만 분석할까요? (Enter = 예 / n = 그대로 두기): 2
→ 대상 5개 구 중 1개 구만 자료 있음 (자료 없음: 종로구, 중구, 광진구, 강서구) → 대상 구 관악구 로 분석 (mapping 의
target_gu. 원래 대상 구는 target_gu_all) 2
AREA_BB0X = None ← 자료가 대상 범위 안쪽(넘는 폭 2km 미만)이라 좁히지 않음 3

...

bld_use          용도          100% 건물 레이어의 용도 칸 4

...

road_mtrwy       자동차전용          100% 도로중심선의 자동차전용 칸 [v6.1]
parcel_id        pnu          100% 필지 고유번호(19자리) 칸

...

쓸 수 있는 방식: layer → register → all (이 순서로 check.py 가 표본 연결률을 재서 처음으로 80% 를 넘는 방식
을 고름) 5

...

Enter = auto, 직접 정하려면 입력:

...

다음: python check.py
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

① 11개 폴더 모두 O, ② 1개 구만 자료 있음, 끝에 다
음: **python check.py**

✗ 이상하면

② 에서 n 을 쳤으면 그대로 진행 → check 가 같은 질문을
다시 함 (단계 4)

단계 4. 출발 전 점검 (check)

이 단계에서 할 일

분석 전에 자료가 말이 되는지 봅니다 (몇 분). 맨 아래 === 결과 === 만 먼저 봅니다.

1. 아래 상자를 칩니다.
2. 그림 ① 통과 → 단계 5 로. ! 경고 줄은 읽고 그대로 진행해도 됩니다. 오늘은 그림 ② 대상 5개 구 중 1개 구만 자료 있음 경고가 **정상**입니다.
3. 그림 ③ 대상 구 관악구: 면적의 ...% 가 자료 범위 안 이 거의 100% 인지 봅니다.
4. 그림 ④ [자동 선택] : 건물 용도를 붙이는 방식을 layer → register → gisbld → all 순서로 재서 **처음으로 80% 를 넘는 방식**을 고릅니다. 1:5,000 건물에는 용도 칸이 있어 **layer** 가 골릴 것입니다. layer 연결률 ...% 와 → 고른 방식: 을 메모 칸에 적습니다. layer 가 80% 아래이면 바로 아래 정의서 코드·코드명으로 읽히지 않은 용도 값 ... (글자 모양: ...) 줄도 그대로 적습니다 (다음 번들에서 맞춤).
5. 그림 ⑤ 관악구의 주거 비율과 주거→길 연결 을 메모 칸에 적습니다.

하는 일 출발 전 점검 (자료는 읽기만 함, 결과는 output\check_report.txt 에도 저장)

입력 python check.py

check 화면 (가운데 줄임)

```
OSGeo4W Shell
건물 용도 방식: auto, 대상 구: 관악구, 옆 구: 없음
...
! 경고: 대상 5개 구 중 1개 구만 자료 있음 (자료 없음: 종로구, 중구, 광진구, 강서구) → 관악구 만 분석 (summary·결과
점검표에 남음) ②
→ 나머지 구 자료를 받으면 python setup.py 다시 (원래 대상 구로 되돌림)
대상 구 관악구: 면적의 100% 가 자료 범위 안 ③
...
! 경고: 결과 약화: sidewalk_cl, bus_stop 없음 (길 일부·계단·정류장·역이 빠짐) → 그대로 진행 가능
...
[자동 선택] 순서 layer → register → gisbld → all, 기존 연결률 80% ④
layer 연결률 100% (표본 건물 40) → 고른 ④
→ 고른 방식: layer (표본 연결률 100% ≥ 80%) - mapping.txt 의 building_attr_chosen 에 적음. 바꾸려면
building_attr_mode 를 직접 정하기
관악구: 건물 40, 주거 33 (82%), 의료 4, 노유자 0, 주거→길 연결 100% ⑤
...
=== 결과 ===
통과 (경고 2건) → python run_all.py --modes auto,all ①
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ **성공하면**

① 통과

✗ **이상하면**

!! 멈춤 → 단계 4-2

단계 4-2. check 가 멈췄을 때: 질문에 Enter 또는 mapping.txt 고치기

이 단계에서 할 일

!! 멈춤: 줄과 그 아래 → 줄을 읽습니다. 대상 구가 자료 범위 밖 이면 check 를 다시 치고 질문에 Enter 하면 됩니다. 그 밖에는 대부분 mapping.txt 한 줄을 고칩니다.

- 그림 ① 이 묻는 줄, ② 가 멈춘 이유, ③ 이 할 일입니다. 아래 예는 setup·check 의 "관악만 분석할까요?" 에 n 을 쳤을 때입니다.
- python check.py** 를 다시 치고, 그림 ① 질문에 Enter → 관악구만 분석하도록 mapping.txt 에 적고 통과합니다.
- mapping.txt 를 고칠 때는 **notepad mapping.txt** 로 열어 Ctrl+F 로 키를 찾고, = 오른쪽만 고쳐 저장합니다 (그림 ④, # 뒤 설명은 그대로). 고친 뒤 **python check.py** 를 다시.

멈춤 예: "관악만 분석할까요?" 에 n 을 쳤을 때

```
OSGeo4W Shell
자료가 덮는 대상 구: 관악구 (대상 5개 구 중 1개 구만 자료 있음 (자료 없음: 종로구, 중구, 광진구, 강서구))
관악구 로 분석할까요? (Enter = 예 / n = 그대로): ①
...
!! 멈춤: 대상 구가 자료 범위 밖: 종로구, 중구, 광진구, 강서구 ②
→ 자료가 덮는 구만 분석하려면 python check.py 다시 → 질문에 Enter. 그 구 자료가 있어야 하면: 수치지형도 폴더가
map_folders 에 있는지 (python setup.py 다시), 받은 자료에 없으면 담당자에게 요청 ③
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

mapping.txt (관악만 분석하기로 한 뒤. 고칠 때는 = 오른쪽만)

```
mapping.txt - 메모장
파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

target_gu = 관악구 # 결과(격자·행정동·요약)를 낼 구. 비우면 자료 전체
target_gu_all = 종로구,중구,관악구,광진구,강서구 # [v6.1] 원래 대상 구: 자료가 일부 구만 덮어 target_gu
를 줄였을 때 setup·check 가 적음 (자료를 더 받고 setup 을 다시 하면 이 구들로 되돌림)
building_attr_mode = auto ④ # 건물 용도·층수를 어디서: auto(check 가 연결물로 고름) / layer(건물
칸) / register(건축물대장) / gisbld(GIS건물통합정보) / all(모든 건물을 집) / stop
walk_exclude_mtrwy = MWI002 # 네트워크에서 뺀 자동차전용 값 (MWI002 = 자동차전용. 자료 값이 Y·N 같은
다른 형식이면 그 값을 적음)
parcel_id = pnu # 필지 고유번호(19자리) 칸
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

멈춤 줄	고칠 곳
대상 구가 자료 범위 밖: ∞구	python check.py 다시 → 질문에 Enter (자료가 덮는 구만 분석). 그 구 자료가 있어야 하는데 없으면 담당자에게
건물 레이어에 용도 칸이 없음	building_attr_mode = register (건축물대장이 없으면 all)
주거 비율 0% 또는 100%	building_attr_mode, bld_use 확인
주거 건물 → 길 연결 ...% (50% 미만)	길 폴더가 빠졌는지 setup 표 확인, layer_road_cl
필지 고유번호 칸을 찾지 못함	parcel_id = 뒤를 필지 칸 중 19자리 번호 칸 이름으로

단계 5. 분석 (run_all)

이 단계에서 할 일

분석을 두 방식으로 한 번에 돌립니다: check 가 고른 방식(auto)과 모든 건물을 집으로 보는 방식(all). 하나가 이상해도 다른 하나로 결과를 가지고 나올 수 있게 하는 것입니다. 멈추면 처음부터가 아니라 멈춘 번호부터 다시 합니다(단계 6).

1. 아래 상자를 치고 Enter. 끝날 때까지 창을 닫지 않고, 창 안을 클릭하지 않습니다.
2. 맨 위 그림 ① 줄에 대상 구: 관악구·건물 용도 방식(두 개)·AREA_BB0X 가 setup·check 대로인지, 그림 ②에 대상 5개 구 중 1개 구만 자료 있음 이 있는지 봅니다. check 가 all 을 골랐으면 # auto = all 이라 한 번만 실행 이 나오고 한 번만 돕니다.
3. 관악 11도엽을 두 방식으로 계산합니다 (02 길 만들기는 한 번, 03 부터 방식마다). 걸리는 시간: 약 3~6분 (1차 방문 실측 01~05 약 1분·06 약 30초를 두 방식으로 어림. DEM 1m 비교(08)가 더해져 조금 더 걸릴 수 있음). 새 글자가 한동안 안 나와도 정상. 30분 넘게 변화가 없으면 메모하고 담당자에게.

하는 일 분석 01 → 02 → 03 → 04 → 05 (+ 필지·상호제공·DEM 1m 이 있으면 06·07·08·10·11), 03 부터는 두 방식

입력 `python run_all.py --modes auto,all`

실행 첫 줄

```
OSGeo4W Shell
C:\Users\user\Desktop\박수범\hbi6>python run_all.py --modes auto,all
# run_all.py --modes auto,all / 단계: 01, 02, 03, 04, 05, 06, 07, 08, 10, 11 / 대상 구: 관악구 / 건물
용도 방식: layer,all / AREA_BB0X: None ①
# 대상 구 범위: 대상 5개 구 중 1개 구만 자료 있음 (자료 없음: 종로구, 중구, 광진구, 강서구) → 관악구 만 분석 ②
# 자료 구조 저장 → output/schema/schema.txt

##### 01_inspect.py #####

##### 03_hbi.py [방식 layer] #####

##### 03_hbi.py [방식 all] #####
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ **성공하면**

줄이 01, 02, 03 ... 순서로 나타남

✗ **이상하면**

!! ... 에서 중단 → 단계 6

단계 5-2. 분석 끝난 뒤

이 단계에서 할 일

숫자를 메모 칸에 적고 결과가 다 나왔는지 봅니다.

1. 맨 아래에 `C:\Users\user\Desktop\박수범\hbi6>` 가 다시 보이면 끝. 화면 어디에도 **!! ...** 에서 중단 이 없어야 합니다.
2. 맨 끝 **=== 결과 점검표 ===** (방식마다 하나씩)와 **=== 방식별 결과 폴더 ===** 를 봅니다. 두 방식 모두 빨강이 없으면 반출. 한 방식만 빨강이면 그대로 반출해도 됩니다 (빨강 없는 방식의 결과를 씀). [빨강] 이 있으면 그 아래 → 대로 고치고 → 고친 뒤: 의 명령을 그대로 칩니다 (`output\방식\result_check.txt` 에도 저장, 기준은 `docs/결과점검_기준.md`). 오늘은 결과 점검표 맨 위에 그림 ⑤ 대상 구 범위 줄이 있고, 선정지는 계산 1/5곳 (봉천동만)이 정상입니다.
3. 마우스 휠로 위로 올려 아래 줄을 찾아 메모 칸에 적습니다.

- 그림 ① 02 의 도로구분(...) 값: 과 자동차전용(...) 값: 두 줄 (글자 그대로. 값 형식을 다음 번들에서 맞춤). 그 아래 ▲ ... 정의서 코드·코드명이 아닌 값 줄이 있으면 그것도

- 그림 ② 걸을 수 없는 길(고속국도·자동차전용) 링크 수, 남긴 연결망과 노드 비율 (90% 이상이면 정상)

- 그림 ③ 대상 구 안 주거 수 / 읽은 주거 수

- 그림 ④ 완료 → `output/` (반출 신청 대상)

- 그림 ⑥ 방식마다 빨강이 있는지, 선정지 봉천동 백분위 (방식마다 하나씩, 고른 방식과 all 둘 다)

4. 방식마다 한 번 나오는 **!! 구간 대현산배수지공원(...): ... 건너뛴** 은 관악 밖 구간이라 정상.

위로 올려서 보는 부분 (리허설: 가짜 자료. ⑤ 는 고른 방식, ⑥ 백분위는 all 점검표)

```
OSGeo4W Shell
[17:18:05] 도로구분(도로구분) 값: RDC014 소로 157, RDC001 고속국도 5, RDC004 특별시도 4 ← 빨 값 RDC001 ①
[17:18:05] 자동차전용(자동차전용) 값: MWI001 일반 157, MWI002 자동차전용 9 ← 빨 값 MWI002 ①
...
[17:18:06] 걸을 수 없는 길 링크 544개를 뺌 (전체 11,730개의 4.6%, 선 9개) ② - 비교용으로 빠지 않은 네트워크 (v5 방식)도 함께 만들
...
[17:18:06] 끊긴 끝점 연결 0개 → 남긴 연결망 1개 (노드 수 [10679]), 노드 비율 100.0% ② (90% 이상이면 정상. 떨어진 구 수만큼 연결망이 나오는 것이 정상)
...
[17:18:08] 대상 구(관악구) 안 주거 116개 / 읽은 주거 135개 ③ (나머지는 출발점에서 뺌, 목적지·길로는 씀)
...
[17:18:12] 완료 → output/ (반출 신청 대상) ④
...
· 대상 구 범위: 대상 5개 구 중 1개 구만 자료 있음 (자료 없음: 종로구, 중구, 광진구, 강서구) → 결과는 관악구 만 ⑤
...
[노랑] 선정지 백분위: 계산 1/5곳, 70% 이상 0곳 (봉천동 25%) ⑥
...
=== 방식별 결과 폴더 ===
output/layer/ → 빨강 없음 ⑥
output/all/ → 빨강 없음 ⑥
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

④ 가 있고 중단 없음 (건너뛴 은 정상)

✗ 이상하면

② 노드 비율이 90% 미만이면 메모하고 계속. ① 에 (정의서에 없는 값) 이 있으면 그 줄을 그대로 적고 계속 (그 값은 이번에는 빠지 않음)

단계 6. 멈췄을 때: 그 번호부터 다시 (--from)

이 단계에서 할 일

!! 원인: · !! 할 일: 을 읽고 고친 뒤, 멈춘 번호부터 다시 돌립니다 (앞 단계 결과는 그대로 씀).

1. 그림 ① 이 멈춘 단계 번호, ② 가 할 일, ③ 이 다시 칠 명령입니다. 단계 5 의 `--modes auto,all` 은 기억해 두므로 `--from` 만 쳐도 두 방식으로 이어서 돕니다.
2. 할 일대로 고칩니다 (mapping.txt 면 단계 4-2 처럼 `notepad mapping.txt` 고친 뒤 `python check.py`).
3. 그림 ③ 의 명령을 그대로 칩니다. 아래 상자는 06 에서 멈췄을 때의 예.
4. 표에 없는 오류이면 화면 마지막 줄을 메모 칸에 적습니다 (output\diagnose.txt 에도 저장됨).

하는 일 06 단계부터 다시 (멈춘 번호를 넣음. 02 는 오래 걸리므로 다시 하지 않음)

입력 `python run_all.py --from 06`

멈춘 예: 필지 칸 이름을 잘못 고쳤을 때

```
OSGeo4W Shell
##### 06_parcel.py [방식 layer] #####
필지에 고유번호 칸 PNU_NO 가 없습니다 → mapping.txt 의 parcel_id 를 실제 칸 이름으로 (python setup.py 가 찾아 줌)
!! 06_parcel.py 에서 중단. 위 오류 메시지를 확인하세요. 1
!! 원인: 위 줄에 멈춘 이유를 적어 둠
!! 할 일: 위 줄의 → 뒤 안내대로 고치기 (mapping.txt 면 python check.py 로 확인) 2
!! 고친 뒤: python run_all.py --from 06 3
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

다시 친 뒤 완료 → output/ 까지 나옴

✗ 이상하면

같은 곳에서 또 멈추면 단계 9 표, 그래도 안 되면 마지막 줄을 적고 담당자에게

단계 7. 실측 좌표 넣기와 추가 분석

이 단계에서 할 일

토요일 현장실측 기록지의 시설 양 끝 좌표를 넣고, 시설 효과(09)와 추가 분석(12·13·14)을 돌립니다.

1. **notepad external\interventions.csv** 로 엽니다. 시설 줄마다 **a_lon,a_lat,b_lon,b_lat** 자리(쉼표 사이)에 기록지의 **아래 끝(a)·위 끝(b)** 경도·위도를 적습니다 (경도 126~127, 위도 37). 저장할 때 인코딩은 **UTF-8** 그대로. 오늘(관악만) 계산되는 시설은 **봉천동 비안어린이공원 인근**(집에서 위성 보기로 찍은 좌표)뿐입니다. 토요일에 잔 대현산·청구동은 중구라, 넣어 두어도 오늘은 건너뛴다 (중구 자료가 들어온 날 계산). 실측 구간(od_pairs.csv)도 토요일 구간이 모두 중구라 오늘은 넣지 않습니다.
2. 아래 첫 상자: 좌표가 다 찬 시설이 있으면 09 가 자동으로 들어갑니다 (02~05 는 다시 안 함).
3. 나머지 세 상자를 하나씩. 13 은 좌표가 있는 시설만 계산하고, 빈 시설은 건너뛴 으로 적습니다. **--modes** 로 돌렸으면 check 가 고른 방식 결과로 돕니다 (첫 줄에 표시, 다른 방식은 명령 뒤에 **--mode all**).
4. 오늘은 관악 밖 지점이 → 길 자료 범위 밖으로 보고 건너뛴 이 붙은 줄로 나옵니다 (그림 ①, 정상. 09 는 끝점이 가장 가까운 길에서 ..., 거리 대신 5km 넘게 일 수 있음). 12 는 등시선 점 5곳 중 **봉천동만** 계산해 그림 ② (9행) 이 됩니다. 09 는 좌표가 찬 시설이 모두 관악 밖이면 **09 결과 없음** 으로 끝납니다 (멈춘 것 아님). 단, **봉천(봉천동·비안어린이공원)**이 **건너뛴 줄에 나오면 좌표 오타입니다** → 기록지와 대조 (경도 126.9..., 위도 37.4...) 후 **python run_all.py --from 09** 다시.
5. **14 는 맨 마지막에**칩니다. 14 를 친 뒤 좌표를 고쳐 다시 돌려야 하면 **python run_all.py --from 03**부터 치고 12·13·14 를 다시칩니다 (**--from 09**로 치면 결과 점검표에 주거 비율: - 빨강이 잘못 나옴).

하는 일 좌표를 넣은 시설 효과(09)부터 다시 (앞 결과는 그대로 씀)

입력 **python run_all.py --from 09**

하는 일 시설 주변 걸어서 갈 수 있는 범위

입력 **python 12_isochrone.py**

하는 일 시설 둘레 땅 종류 (좌표 있는 시설만)

입력 **python 13_siting.py**

하는 일 행정동별 동네 성격

입력 **python 14_dong_context.py**

세 줄의 끝 화면 (리허설: 좌표 넣기 전이라 13 은 건너뛴)

```
OSGeo4W Shell
[17:18:27] !! 창신역 1번출구: 가장 가까운 길이 9621m 떨어져 있음 → 길 자료 범위 밖으로 보고 건너뛴 (이 점은 결과 없음) 1
***
[17:18:27] 완료 → output/isochrone_*.gpkg, isochrone_summary.csv (9행) 2, 길 자료 범위 밖이라 건너뛴
4곳: 창신역 1번출구, 대현산배수지공원 모노레일 하부, 신당동 청구동 마을마당, 중국동 용곡중
***
external/interventions.csv 에 a·b 좌표가 모두 있는 행이 없습니다 → 건너뛴
***
[17:18:29] 완료 → output/dong_context.csv (4행, 건물·셀 좌표는 저장하지 않음)
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

--from 09 에 09_intervention.py 가 나오고 봉천 시설 수해 건물 ... 줄, 12·14 완료 → (관악 밖 건너뛴 은 정상). 끝의 빨강 없음 은 결과 점검을 다시 한 것이 아님 (단계 5-2 의 점검표가 기준)

✗ 이상하면

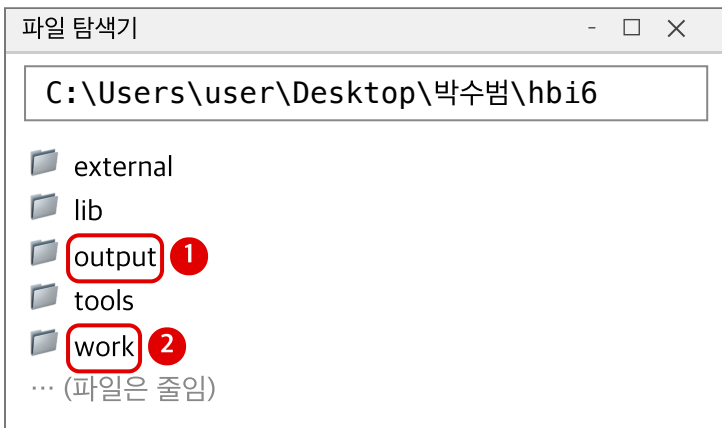
--from 09: 이번 실행 목록에 없는 단계 → 좌표 네 칸 중 빈 칸이 있음 (기록지 확인). 봉천이 건너뛴 → 좌표 오타. 14 뒤에 다시 돌려 주거 비율: - 빨강 → --from 03 부터

단계 8. 반출과 마무리

이 단계에서 할 일

가지고 나갈(반출할) 폴더를 확인합니다.

1. 반출할 것: **hbi6** 폴더 안의 **output** 폴더 하나 (그림 ①). 두 방식 결과가 `output\고른방식`
`\·output\all\` 에 모두 들어 있어 **둘 다 함께** 나갑니다. 방식 폴더마다 결과 CSV·지도
· `result_check.txt` · `run_meta.csv`, `output` 맨 위에 `check_report.txt` · `runlog` · `schema`
가 있습니다 (값 없이 구조·비율·개수만).
2. 반출하지 않을 것: **work** (그림 ②, 건물 하나하나의 결과와 풀린 zip 이 있어 반출 금지). `external`, `lib`,
`tools`, `mapping.txt` 도 그대로 둡니다.
3. 담당자에게 `output` 폴더 반출 신청 (심사 3~4일).



화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

✓ 성공하면

`output` 반출 신청, 메모 칸이 채워짐

✗ 이상하면

`output` 에 `diagnose.txt` 가 있으면 마지막 실행
이 실패한 것 → 단계 6

단계 9. 막혔을 때

이 단계에서 할 일

화면에 나온 글자로 아래 표에서 찾아 그대로 합니다.

화면에 나온 것	뜻	할 일
'python'은(는) 내부 또는 외부 명령...이 아닙니다	일반 명령창에 쳤음	단계 1 처럼 OSGeo4W Shell 을 다시 열고 <code>cd /d C:\Users\user\Desktop\박수범\hbi6</code>
can't open file ... No such file or directory	hbi6 폴더 밖	<code>cd /d C:\Users\user\Desktop\박수범\hbi6</code> 로 이동 후 다시
창 제목 앞에 선택	창 안을 클릭해 멈춤	Esc
ㅅㅌㅅㅌㅌㅌ 처럼 한글로 쳐짐	한글 입력 상태	지우고 한/영 키 한 번
화면 한글이 깨짐 (???·이상의 글자)	명령창 글자 형식(코드 페이지)	<code>chcp 65001</code> 치고 같은 명령 다시 (분석 결과·저장 파일은 영향 없음)
!! 멈춤: 대상 구가 자료 범위 밖 (check)	관악만 분석할지 묻는 질문에 n 을 쳤음	<code>python check.py</code> 다시 → 질문에 Enter (단계 4-2)
!! 멈춤: (check, 그 밖)	자료와 설정이 맞지 않음	단계 4-2
!! ... 에서 중단 (run_all)	그 단계에서 멈춤	단계 6 (--from 번호)
SyntaxError 와 utf-8	메모장이 UTF-8 이 아닌 형식으로 저장	메모장 → 파일 → 다른 이름으로 저장 → 인코딩 UTF-8 → 같은 이름으로

`config.py` 의 `EDGE_BUFFER` 줄은 바꾸지 않습니다. 표에 없는 오류이면 오류 마지막 줄을 메모 칸에 적고 담당자에게 알립니다.

오류 예시: 일반 명령창(명령 프롬프트)에 쳤을 때

명령 프롬프트

```
C:\Users\user\Desktop\박수범\hbi6>python run_all.py
'python'은(는) 내부 또는 외부 명령, 실행할 수 있는 프로그램, 또는
배치 파일이 아닙니다. 1
```

화면 예시 — 실제 화면과 글꼴·색·숫자가 다를 수 있음

2차 방문 메모 칸

이 단계에서 할 일

아래 칸을 채워 옵니다. 데이터 값은 적지 않습니다 (이름·개수·비율·코드 분포만). ★ 는 다음 번들(나머지 구 분석용)을 맞추는 데 꼭 필요한 줄입니다.

항목	적을 것
단계 0 ④ 나머지 4개 구 자료	들어옴 / 아직
★ 박수범 폴더 안 폴더 이름 (탐색기)	DEM 5m: ____ / DEM 1m: ____ / 상호제공: ____
단계 3-2 관악만 분석	예 (Enter) / 5개 구 그대로
단계 4 고른 방식 · layer 연결률	layer / register / all · ____ %
★ 단계 4 읽히지 않은 용도 값 줄 (있으면 그대로)	
단계 4 관악구 주거 비율 · 길 연결	____ % · ____ %
★ 단계 5-2 도로구분(...) 값: 줄 (그대로)	
★ 단계 5-2 자동차전용(...) 값: 줄 (그대로)	
단계 5-2 걸을 수 없는 길 링크 · 남긴 연결망 · 노드 비율	____ 개 · ____ 개 · ____ %
단계 5-2 대상 구 주거 수 / 읽은 주거 수 · 봉천동 백분위 (고른 방식 / all)	____ / ____ 개 · ____ % / ____ %
단계 5 걸린 시간	시작 __:__ → 끝 __:__
오류 마지막 줄 (있었으면)	
반출 신청한 폴더	output (신청 시각 __:__)